

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверила:

Андренко К.В.

Цель работы: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Ход работы

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
9	CIFAR-100	32X32	Adam

Код программы:

```
import os
import random
from collections import defaultdict

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms

SEED = 42
MEAN = (0.5071, 0.4865, 0.4409)
STD = (0.2673, 0.2564, 0.2762)
BATCH_SIZE = 128
EPOCHS = 8
NUM_WORKERS = 2 # на Windows можно поставить 0, если проблема останется

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=100):
        super().__init__()
```

```

self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
self.bn1 = nn.BatchNorm2d(64)
self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.bn2 = nn.BatchNorm2d(128)
self.conv3 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
self.bn3 = nn.BatchNorm2d(256)
self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

self.fc1 = nn.Linear(256 * 4 * 4, 512)
self.fc2 = nn.Linear(512, num_classes)
self.dropout = nn.Dropout(0.5)

def forward(self, x):
    x = self.pool(F.relu(self.bn1(self.conv1(x))))
    x = self.pool(F.relu(self.bn2(self.conv2(x))))
    x = self.pool(F.relu(self.bn3(self.conv3(x))))
    x = torch.flatten(x, 1)
    x = F.relu(self.fc1(x))
    x = self.dropout(x)
    x = self.fc2(x)
    return x

def denorm(img_tensor):
    img = img_tensor.cpu().numpy().transpose(1, 2, 0) * np.array(STD) + np.array(MEAN)
    return np.clip(img, 0, 1)

def show_prediction(model, input_tensor, classes, true_label=None, title_prefix=""):
    model.eval()
    with torch.no_grad():
        probs = F.softmax(model(input_tensor.to(next(model.parameters()).device)),
dim=1) \
                .cpu().numpy().flatten()
        pred_label = int(np.argmax(probs))

        top5_idx = np.argsort(probs)[-5:][::-1]
        top5_names = [classes[i] for i in top5_idx]
        top5_probs = [probs[i] * 100 for i in top5_idx]

        fig, axes = plt.subplots(1, 2, figsize=(12, 4), gridspec_kw={'width_ratios': [1,
1.6]})

        axes[0].imshow(denorm(input_tensor[0]))
        axes[0].axis('off')
        if true_label is not None:
            correct = pred_label == true_label
            axes[0].set_title(
                f"{title_prefix}\nИстина: {classes[true_label]}\n"
                f"Предсказание: {classes[pred_label]}",
                color='green' if correct else 'red'
            )
        else:
            axes[0].set_title(f"{title_prefix}\nПредсказание: {classes[pred_label]}")

        colors = ['green' if true_label is not None and classes[i] == classes[true_label]
            else ('red' if i == pred_label else 'steelblue') for i in top5_idx]

        axes[1].barh(top5_names, top5_probs, color=colors)
        axes[1].set_xlabel("Вероятность, %")
        axes[1].set_xlim(0, 100)
        axes[1].invert_yaxis()
        axes[1].grid(alpha=0.3, axis='x')

```

```

plt.tight_layout()
plt.show()
return pred_label, probs

def predict_custom_image(model, classes, path_or_pil):
    if isinstance(path_or_pil, str):
        img = Image.open(path_or_pil).convert('RGB')
    else:
        img = path_or_pil.convert('RGB')

    preprocess = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])
    input_tensor = preprocess(img).unsqueeze(0)
    pred_label, probs = show_prediction(model, input_tensor, classes, true_label=None,
                                       title_prefix="Пользовательское изображение")

    return pred_label, probs

if __name__ == '__main__':
    # Воспроизводимость
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Используемое устройство:", device)

    transform_train = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])

    transform_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])

    train_set = torchvision.datasets.CIFAR100(
        root='./data', train=True, download=True, transform=transform_train
    )
    test_set = torchvision.datasets.CIFAR100(
        root='./data', train=False, download=True, transform=transform_test
    )

    CLASSES = train_set.classes
    NUM_CLASSES = len(CLASSES)

    train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True,
                              num_workers=NUM_WORKERS)
    test_loader = DataLoader(test_set, batch_size=256, shuffle=False,
                              num_workers=NUM_WORKERS)

    print(f"Train: {len(train_set)}, Test: {len(test_set)}, "
          f"размер изображения: {train_set[0][0].shape}")
    print(f"Классов: {NUM_CLASSES}")

```

```

model = SimpleCNN(num_classes=NUM_CLASSES).to(device)
print(model)
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Обучаемых параметров: {total_params:,}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=5e-4)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)

history = {"train_loss": [], "test_loss": [], "train_acc": [], "test_acc": []}

for epoch in range(1, EPOCHS + 1):
    # --- train ---
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    train_loss = running_loss / total
    train_acc = correct / total

    # --- test ---
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * images.size(0)
            _, preds = outputs.max(1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

    test_loss = running_loss / total
    test_acc = correct / total
    scheduler.step()

    history["train_loss"].append(train_loss)
    history["test_loss"].append(test_loss)
    history["train_acc"].append(train_acc)
    history["test_acc"].append(test_acc)

    print(f"Эпоха {epoch:2d}/{EPOCHS} | "
          f"train loss {train_loss:.4f} acc {train_acc*100:.2f}% | "
          f"test loss {test_loss:.4f} acc {test_acc*100:.2f}%")

epochs_range = range(1, EPOCHS + 1)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(epochs_range, history["train_loss"], label="Train loss", marker='o',
markersize=3)

```

```

axes[0].plot(epochs_range, history["test_loss"], label="Test loss", marker='o',
markersize=3)
axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
axes[0].set_title("Изменение ошибки при обучении")
axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].plot(epochs_range, [a*100 for a in history["train_acc"]], label="Train
acc", marker='o', markersize=3)
axes[1].plot(epochs_range, [a*100 for a in history["test_acc"]], label="Test
acc", marker='o', markersize=3)
axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("Accuracy, %")
axes[1].set_title("Изменение точности при обучении")
axes[1].legend(); axes[1].grid(alpha=0.3)

plt.tight_layout(); plt.show()
print(f"Итоговая точность на тесте: {history['test_acc'][-1]*100:.2f}%")

model.eval()
class_correct, class_total = defaultdict(int), defaultdict(int)
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images).argmax(1)
        for label, pred in zip(labels, preds):
            class_total[label.item()] += 1
            class_correct[label.item()] += int(label.item() == pred.item())

print("\nТочность по классам (топ-10 лучших и топ-10 худших):")
accs = [(CLASSES[i], 100 * class_correct[i] / class_total[i]) for i in
range(NUM_CLASSES)]
accs_sorted = sorted(accs, key=lambda x: x[1], reverse=True)
print("  Лучшие:")
for name, acc in accs_sorted[:10]:
    print(f"    {name:>20s}: {acc:5.2f}%")
print("  Худшие:")
for name, acc in accs_sorted[-10:]:
    print(f"    {name:>20s}: {acc:5.2f}%")

print("\nСлучайные примеры из тестовой выборки:")
for _ in range(3):
    idx = random.randint(0, len(test_set) - 1)
    image, true_label = test_set[idx]
    show_prediction(model, image.unsqueeze(0), CLASSES,
                    true_label=true_label, title_prefix=f"Пример #{idx}")

predict_custom_image(
    model, CLASSES,
    os.path.join(os.path.expanduser("~"), "Desktop", "test1.png")
)

```

Результат программы:

Используемое устройство: cpu

Train: 50000, Test: 10000, размер изображения: torch.Size([3, 32, 32])

Классов: 100

SimpleCNN(

(conv1): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)

```
(conv2): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
(conv3): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, bias=True,
track_running_stats=True)
(pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(fc1): Linear(in_features=4096, out_features=512, bias=True)
(fc2): Linear(in_features=512, out_features=100, bias=True)
(dropout): Dropout(p=0.5, inplace=False)
)
```

Обучаемых параметров: 2,520,676

Эпоха 1/8	train loss 4.0600 acc 7.21%	test loss 3.5327 acc 15.55%
Эпоха 2/8	train loss 3.6259 acc 12.88%	test loss 3.1609 acc 21.59%
Эпоха 3/8	train loss 3.3870 acc 16.79%	test loss 3.0008 acc 24.97%
Эпоха 4/8	train loss 3.1949 acc 20.08%	test loss 2.8330 acc 28.01%
Эпоха 5/8	train loss 3.0570 acc 22.66%	test loss 2.6321 acc 32.55%
Эпоха 6/8	train loss 2.9237 acc 25.38%	test loss 2.4757 acc 35.78%
Эпоха 7/8	train loss 2.8351 acc 27.17%	test loss 2.3842 acc 38.13%
Эпоха 8/8	train loss 2.7689 acc 28.43%	test loss 2.3432 acc 38.54%

Итоговая точность на тесте: 38.54%

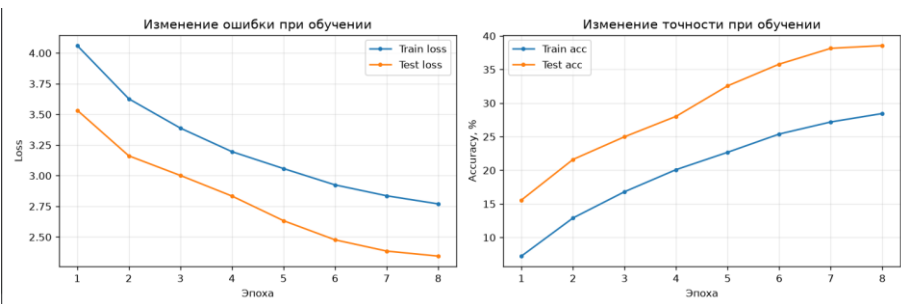
Точность по классам (топ-10 лучших и топ-10 худших):

Лучшие:

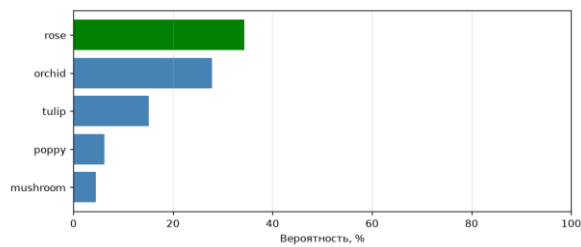
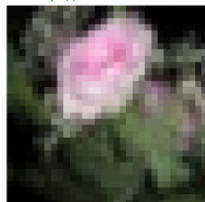
- wardrobe: 84.00%
- motorcycle: 83.00%
- oak_tree: 82.00%
- road: 79.00%
- plain: 77.00%
- sunflower: 77.00%
- castle: 76.00%
- orange: 74.00%
- skunk: 74.00%
- chair: 68.00%

Худшие:

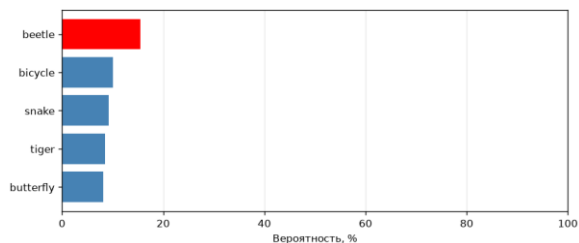
- kangaroo: 13.00%
- snail: 12.00%
- tulip: 12.00%
- turtle: 12.00%
- mouse: 11.00%
- seal: 11.00%
- rabbit: 10.00%
- possum: 8.00%
- lizard: 7.00%
- otter: 3.00%



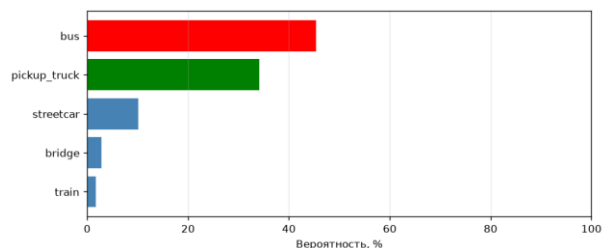
Пример #1824
Истина: rose
Предсказание: rose



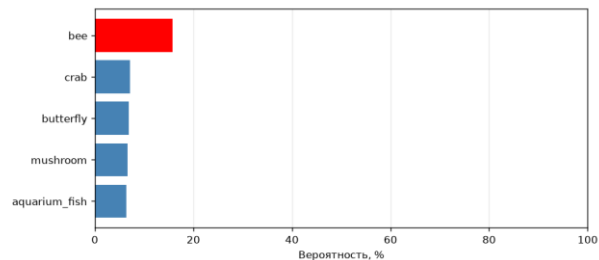
Пример #409
Истина: tractor
Предсказание: beetle



Пример #4506
Истина: pickup_truck
Предсказание: bus



Пользовательское изображение
Предсказание: bee



Простая свёрточная сеть из базовых слоёв способна обучаться на CIFAR-100 и демонстрирует правильную динамику: ошибка падает, точность растёт. Однако достигнутый уровень (~27–40% на тесте за 3–7 эпох) далёк от результатов современных архитектур. Сеть путает визуально похожие классы (например, rose → orchid, tulip) и плохо справляется с изображениями, непохожими по стилю на обучающую выборку. Чтобы приблизиться к state-of-the-art результатам (~90%+), нужны конкретные

усложнения: более глубокая архитектура (ResNet, DenseNet), приёмы против затухания градиентов (BatchNorm, skip-connections), более длительное обучение (200+ эпох), сильная аугментация (CutMix, MixUp) и более разнообразные данные.

Вывод: проделав данную лабораторную работу я научилась конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения